

Technical Assessment – Junior Full Stack Engineer

Introduction

We are **Liwaza** (<https://liwaza.org>).

We build AI-powered products and automation solutions in education across Africa.

We are looking for a Junior Full Stack Engineer who is capable of learning quickly, reasoning clearly, leveraging modern AI tools effectively, and building production-quality software.

This assessment is intentionally designed for the AI era.

We fully expect candidates to use modern AI tools such as ChatGPT, Claude, Gemini, Cursor, Windsurf, GitHub Copilot, or similar solutions.

The goal of this assessment is not to determine whether AI was used.

The goal is to evaluate:

- your engineering fundamentals;
- your ability to learn and adapt;
- your reasoning and decision-making;
- your product thinking;
- your ability to build modern AI-native systems;
- your ability to verify and improve AI-generated outputs.

A candidate who effectively uses AI while demonstrating strong engineering judgment will score higher than a candidate who avoids AI but produces a weaker solution.

Assignment

For the purposes of this assignment, we will build an AI-native eGov MCP system.

You can learn more about MCP through the link:

<https://modelcontextprotocol.io/docs/getting-started/intro>

Candidates based in Abidjan

Choose one public service that exposes a documented or publicly accessible API.

Examples:

- FNE (Facture Normalisée Électronique)
- DGI Côte d'Ivoire
- GUCE Côte d'Ivoire

Candidates based in Yaoundé

Choose one public service that exposes a documented or publicly accessible API.

Examples:

- DGI Cameroon
- Open Data Cameroon
- GUCE Cameroon

Objective

Build an AI-native eGov platform powered by MCP (Model Context Protocol).

Users should be able to interact with government services through natural language and receive useful, actionable results.

The platform must include:

- a React frontend acting as an MCP client;
- a Python MCP server acting as the backend;
- integration with at least one real public service API;
- deployment to a publicly accessible environment.

The final product should demonstrate how a modern AI-powered public services platform could operate.

Part 1 – MCP Server (Backend)

Required Stack

- Python
- FastAPI
- Pydantic

The MCP server is the backend of the application.

The MCP server must expose at least five useful tools.

Examples:

- retrieve obligations;
- verify registrations;
- search records;
- retrieve deadlines;
- calculate contributions;
- submit declarations;
- retrieve public datasets.

Requirements:

- validation;
- error handling;
- authentication;
- logging;
- documentation.

You must explain why you designed the MCP tools the way you did.

Part 2 – Frontend

Required Stack

- React.js

You may additionally use:

- Next.js
- TypeScript

- Tailwind
- Material UI
- shadcn/ui

or any other technologies you can justify.

AI-Native Experience

The primary interaction model must be conversational.

Users should be able to ask questions in natural language in either English or French.

Examples:

French:

- Prépare ma déclaration TVA de mars
- Vérifie les échéances fiscales
- Génère la DSF à partir de mes écritures
- Vérifie ce matricule CNPS
- Fais le calcul automatique de mes cotisations sociales pour mes employés

English:

- Prepare my March VAT declaration
- Check my upcoming tax deadlines
- Generate the annual tax filing from my accounting records
- Verify this CNPS registration number
- Calculate social contributions for my employees

The system should:

- understand the request;
- identify the appropriate MCP tools;
- execute those tools;
- return useful results.

The frontend should provide a modern AI experience similar to contemporary AI products.

Examples of features:

- conversational interface;
- conversation history;
- tool execution visibility;
- structured outputs;
- error handling.

Design System

Create a lightweight design system.

Include:

- colors;
- typography;
- spacing;
- reusable components;
- responsive layouts.

We will evaluate:

UX

- usability;
- navigation;
- clarity;
- accessibility.

UI

- visual hierarchy;
- consistency;
- polish;
- overall product quality.

We prefer a simple and elegant product over a complex one.

Monorepo vs Multi-Repository

You must explain:

- whether you chose a monorepo or not;
- why;
- advantages;
- disadvantages.

There is no preferred answer.

We care about your reasoning.

Part 3 – MCP Architecture Requirements

The MCP server is the backend.

The React application is the MCP client.

This architecture is mandatory.

Expected architecture:

React Frontend (MCP Client) → MCP Server → Government APIs

The frontend must communicate with the MCP server.

Business logic should live inside the MCP server.

The frontend should not bypass the MCP layer.

Part 4 – Real Tool Execution Requirement

Tool execution must be real.

Hardcoded responses are prohibited.

Fake responses from mock data are prohibited.

You should assume that the reviewers will inspect:

- API traffic;
- MCP traffic;
- tool execution;
- request traces;
- backend behavior.

The purpose is to verify that:

- MCP tools are genuinely executed;
- results come from actual workflows;
- responses are not simulated.

Any attempt to fake tool execution or fabricate results may result in disqualification.

Part 5 – DevOps

The application must be publicly accessible.

The reviewer must be able to:

- access the frontend;
- access the backend API;
- access MCP endpoints;
- review documentation.

Deployment

Use free-tier services.

Examples:

- Vercel
- Render
- Railway
- Fly.io
- Cloudflare

The choice is yours.

You must explain your deployment decisions.

Required Deliverables

- Dockerfile
- docker-compose
- CI/CD workflow
- environment variable management
- deployment documentation

Part 6 – Architecture

Provide an Architecture Decision Document.

Include:

Current Architecture

- architecture diagram;

- service interactions;
- deployment topology;
- data flow.

Scalability

Explain how your solution would evolve from:

100 users

to

100,000 users.

Discuss:

- scaling strategy;
- caching;
- background jobs;
- observability;
- database considerations;
- cost considerations.

Part 7 – AI & LLM Strategy

Assume AI features will continue to grow.

Explain which models you would use and why.

At minimum discuss:

- GPT-4.1
- GPT-4o
- Claude Sonnet
- Claude Opus
- Gemini 2.5
- Llama
- Mistral

Discuss:

- quality;
- cost;
- latency;

- privacy;
- security;
- GDPR considerations;
- compliance considerations;
- self-hosting opportunities.

We want recommendations tied to this specific product.

Part 8 – Testing

Include:

- unit tests;
- integration tests;
- test strategy.

Explain:

- what is tested;
- what is not tested;
- why.

Part 9 – Documentation

Provide a complete README.

Include:

- project overview;
- setup instructions;
- deployment instructions;
- architecture overview;
- screenshots;
- assumptions;
- tradeoffs;
- future improvements.

Part 10 – Engineering Walkthrough Video

Record a video in English.

Duration:

10–15 minutes.

The video must cover:

1. Product overview
2. Architecture decisions
3. MCP design
4. Backend design
5. Frontend design
6. DevOps decisions
7. Testing strategy
8. Security considerations
9. AI/LLM strategy
10. Future improvements

We are evaluating how you think, not only what you built.

Time Expectations

This assessment is designed to take approximately: 12–15 hours of focused work.

You are not expected to build a production-complete system.

We are evaluating your prioritization skills and engineering judgment.

Submission Deadline

Submit your work before Wednesday 10th of June, 23:59 GMT time.

Deliverables

Please submit:

- GitHub repository (public)
- Frontend URL
- Backend API URL
- MCP endpoint URL
- Architecture document
- AI strategy document

- Engineering walkthrough video

AI Usage Policy

The use of AI tools is permitted and expected.

Examples:

- ChatGPT
- Claude
- Gemini
- Cursor
- Windsurf
- GitHub Copilot
- Any other AI-assisted development tools

Mandatory Disclosure

You must provide:

- all AI tools used;
- all prompts used during the assessment;
- which parts were AI-assisted;
- which parts were manually written.

Verification Responsibility

You are fully responsible for:

- technical correctness;
- code quality;
- architecture decisions;
- factual accuracy.

Any hallucinated, inaccurate, misleading, or unverified information will negatively impact your score.

AI Detection & Review

All written submissions may be processed through AI-content detection systems and manually reviewed.

This includes:

- architecture documents;
- explanations;
- reports;
- technical justifications;
- AI strategy documents.

The purpose is not to prohibit AI usage.

The purpose is to evaluate:

- critical thinking;
- verification skills;
- engineering judgment;
- ownership of the work.

Candidates who blindly submit AI-generated content without validation will be heavily penalized.

Candidates who effectively use AI while demonstrating understanding and ownership will score significantly higher.

Evaluation Criteria

Area	Weight
Frontend (UX/UI/Design System)	15%
Backend	15%
MCP Design	15%
DevOps	10%
Testing	10%
Documentation	5%
AI Agent & MCP Orchestration	10%
Engineering Reasoning	20%

Total: 100%

Engineering Reasoning (20%)

This is the most important evaluation category.

We want to understand how you think.

We will evaluate:

- decision making;
- tradeoff analysis;
- product thinking;
- scalability thinking;
- security thinking;
- business awareness;
- AI-native engineering practices.

We value engineers who can explain:

"I chose this approach because..."

more than engineers who simply say:

"I used this technology."

A simple solution with strong reasoning will score higher than a complex solution with weak reasoning.